Django Advanced Regular Exam

Individual Project

Project Requirements for **Django Advanced Course @ SoftUni**

Your project is an opportunity to demonstrate your understanding of advanced Django concepts and RESTful API development by building a functional, visually appealing, and production-ready web application. You are expected to apply everything you have learned - from designing models, serializers, and class-based views to creating dynamic templates and maintaining clean, modular code. The project should showcase your ability to design, implement, and deploy a real-world solution independently, following Django's best practices and conventions. Focus on building a cohesive, original concept that reflects both technical proficiency and thoughtful architectural design.

1. Project Requirements

Important Notes:

- The submitted project must be **downloadable**, **installable**, and **runnable without modifications**. After installing dependencies and applying migrations, the application should start successfully using the default configuration.
- All **environment variables and credentials** required for local testing must be clearly documented in your README file.

Core Functional Requirements

- Your application must include **both a public section** (**accessible** to **anonymous** users) and a **private section** (**accessible only to authenticated** users).
    - The application must support **user registration**, **login**, and **logout** functionality.
- There must be **at least two user groups** defined in the admin site, each with **distinct permissions**.
- The **built-in Django User model must be extended**.

Technical Stack

- The application must be implemented using **Django** and/or **Django REST Framework** (latest stable versions).
    - Your project must consist of **at least five** (5) **Django apps**, each with clearly defined responsibilities.
    - The application must define **at least five** (5) **database models**.
        - Models created via inheritance or one-to-one relationships **count as one model** (e.g., User <-> Profile).
        - Your database architecture must include **at least two many-to-one** and **two many-to-many relationships**.

Forms and Validations

- The application must include at least **seven** (7) **forms** with proper **data validations**.

© SoftUni – about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.
Follow us:

- Display **user-friendly error messages** for invalid input.
- Implement **validations** in both models and forms, where appropriate.
- **Customize** error messages, help texts, labels, and placeholders.
- Include **read-only** or **disabled** fields in at least two forms.
- **Exclude** unnecessary fields when rendering forms.
- Provide a **confirmation step** before deleting any object.

Views and APIs

- Implement **views** to handle business logic:
  - Use **Class-Based Views** (CBVs) as the **main approach** (approximately 90% CBVs).
  - Handle forms correctly (GET and POST methods, validation, saving).
  - Use **redirects** after successful create/update actions.
  - Implement **at least one RESTful API endpoint or service** using Django REST Framework.
    - Use serializers, API views, and permissions appropriately.

Templates and Frontend

- The application must include **at least fifteen** (15) **web pages/templates** built using the **Django Template Engine** or rendered by a **front-end framework** (React, Angular, etc.):
  - At least **ten** (10) of them must display **dynamic data** from the **database**.
  - Implement **full CRUD** functionality for at least **three** (3) **models** (manageable by **owners**).
  - Include pages that display **all objects**, **filtered/sorted** objects, and **single-object details**, **user details**, or profiles, etc.
  - Showcase built-in and **custom template filters/tags**.
  - Provide **custom error pages** (404, 500, etc.).
  - Use a **base template** (not included in the 15-count).
  - Apply **template inheritance** and **reusable partial templates**.
  - Ensure that **navigation links connect** all pages **consistently**, and every page **includes** a **footer**.
  - **Display** or **hide links** appropriately for anonymous/authenticated users.
  - Templates can be **reused** across different views when appropriate.
  - Implement a **responsive web design** using **Bootstrap**, an **AI-generated layout**, or a **custom design**.
- All pages in the application **must** be **accessible through navigation links**.
  - Avoid "orphan" pages that can only be reached by typing a URL manually.
  - Ensure consistent navigation menus and footers across the site.
  - **Failure** to provide **functional navigation** will **affect** your score, as any **inaccessible pages** will **not be counted or evaluated**.

© SoftUni – about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.
Follow us:

Additional Technical Requirements

- Implement **asynchronous task processing** (e.g., Celery, RQ, or asyncio-based tasks).
- Ensure application **security** and **data protection**:
  - Prevent SQL injection, XSS, CSRF, and parameter tampering.
  - Use **environment variables** for sensitive data (no hardcoded secrets).
- Use a **PostgreSQL or another** relational Database System.
- **Store** and **serve media** and **static** files properly.
- Include **tests** for **custom logic**, **views**, and **user-related functionality** (minimum 15 tests).
- **Deploy** the project on a **cloud-based platform**.
  - The **functionality** and **overall behaviour** of your project will be **assessed primarily based** on the **deployed version**.
  - Ensure that the **deployed application is fully functional**, synchronized with your GitHub repository, and **accessible during** the **evaluation period**.
- Use **GitHub** as your **version control** platform.
  - **Submit** a **public GitHub repository link**.
  - There must be a **minimum** of **7** (**seven**) **commits** on **7** (**seven**) **separate days**.

Documentation and Code Quality

- Provide clear **project documentation**:
  - A **comprehensive GitHub README** describing setup, dependencies, and deployment.
  - Include environment setup instructions and sample .env configuration.
- Follow **OOP** and **clean code** principles:
  - Apply data encapsulation and proper exception handling.
  - Demonstrate inheritance, abstraction, and polymorphism where relevant.
  - Follow strong cohesion and loose coupling principles.
  - Maintain readable, well-formatted code with clear naming conventions.

Disclaimer

- It is **NOT permissible to use** any of the following in your project:
  - **Ideas**, **Models**, **HTML**, **CSS**, or entire **Django apps** taken from **workshops**, **exercises**, or **lectures/presentations**.
  - **HTML/CSS/JS** from **JS modules**, or any other **SoftUni-related courses**.
  - **AI-generated code** (except for HTML/CSS layouts or documentation).
- If your project is found to be **over 60% AI-generated**, it will be **disqualified** (**0 points**).
- Focus on **developing a unique and original concept** that reflects your **understanding**.

© SoftUni – about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.
Follow us:

- Failure to do so will **negatively affect** your **final grade**.

2. Submission Deadline

- You **must** submit a **GitHub repository link** **before 15:59 on 07 April 2026** using the **submission button** in the **Regular Exam** section, available **from 31 March 2026**.
- You may continue improving your project **after assessment** (**after 16 April 2026**)
  - Do **not push new commits** until **evaluation** is **complete**; work **locally until then**.
- Projects **not submitted by** the **deadline** will **NOT be evaluated**.
- The assessment is performed manually and typically takes up to 10 days.

3. Assessment Criteria

- **Originality** and **Concept** (unique idea, relevance, creativity) **-** maximum **10** points
- **Database Design** and **Relationships** (model structure, M2M, FKs, constraints, fields, methods, Meta, query usage) - maximum **5** points
- **User Model** extension, **Groups/Permissions**, **Authentication** - maximum **5** points
- **Forms**, **Validation**, **Media files** handling - maximum **5** points
- **Views** Implementation (90% CBVs, mixins, advanced techniques) - maximum **10** points
- **Pages** (design, navigation, dynamic data) - maximum **10** points
- **Asynchronous Processing** (task queues, scheduling, background jobs)- maximum **10** points
- **RESTful APIs** (serializers, permissions, integration) - maximum **10** points
- **Deployment** and **Environment Configuration** - maximum **10** points
- **Tests** (coverage, correctness, relevance, minimum 15 tests) - maximum **5** points
- **Security** and **Data protection**, **Advanced Features** and **Functionality** (not demonstrated by lecturer) - maximum **10** points
- Project **Documentation** (README clarity, setup instructions, structure overview) - maximum **4** points
- Version **Control** (Git discipline: commits, structure, descriptive messages) - maximum **3** points
- **Code Quality** and **Architecture** (readability, OOP, structure) - maximum **3** points

© SoftUni – about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.
Follow us: